

RESEARCH FUNDAMENTALS – MINOR PROJECT PROPOSAL

Building a GPT-Style Language Model from Scratch

A transparent, tested, and fully documented implementation of the GPT architecture — from byte-pair tokenization to trained text generation.

Bikram Sharma • Pramesh Lamichhane • Swikar Poudel
Gandaki College of Engineering and Science | 07 JUL 2026

AGENDA

What We Will Cover

01

Literature Review

02

Introduction & Objectives

03

Tools, Methodology & Design

04

Research Plan, Testing & Budget

05

Model Results & Outcomes

06

Contributions & Future Works

CHAPTER 1 · INTRODUCTION

Why Build a GPT from Scratch?

- Most students only ever see the output side of language models — a chat box.
- GPT-style models predict the next token from all previous tokens (Radford et al., 2019).
- Self-attention (Vaswani et al., 2017) replaced recurrence and made this trainable at scale.
- This project builds every component directly — tokenizer, embeddings, attention, training, generation.

Base Repository

GPT-From-scratch

- `config.py` — GPT-124M configuration
- `model/gpt.py`, `model/transformer.py`
- Training + tutorial notebooks
- Sample corpus: `the-verdict.txt`
github.com/beeks-code/GPT-From-scratch

CHAPTER 1 · INTRODUCTION

Problem Statement

“Many learners can use language-model apps, but few can explain — or implement — how tokenization, attention, training, and generation work together.”

Fragmented Tutorials

Tokenization, attention, and training are usually taught as disconnected pieces.

Research Code ≠ Engineering

The base repository works, but lacks tests, configuration logging, and design documentation.

The Gap

No clear, end-to-end, student-built and tested GPT pipeline with proper software documentation.

CHAPTER 1 · INTRODUCTION

Project Objectives

- 1 Study the Transformer/GPT architecture from foundational papers to nanoGPT.
- 2 Implement embeddings, masked multi-head attention, FFN, LayerNorm, and output head in PyTorch.
- 3 Build a GPT-2 BPE tokenized dataset pipeline with context windows.
- 4 Train & validate on a chosen corpus, logging loss, perplexity, and samples.
- 5 Implement temperature and top-k decoding strategies.
- 6 Refactor the repository into a clean, tested, reproducible codebase.
- 7 Produce a full set of UML/ERD/sequence/class design diagrams.

CHAPTER 2 · LITERATURE REVIEW

Literature Review & Gap Analysis

From Vaswani et al.'s original Transformer to nanoGPT-style educational rebuilds, the architecture is well published — but rarely reproduced end-to-end, tested, and documented by the students studying it.

Foundational Papers

Vaswani et al. (2017) introduced self-attention; Radford et al. (2019) scaled it into GPT-2.

Educational Rebuilds

Public rebuilds like nanoGPT share training code, but usually skip tests and design docs.

The Gap

No available example pairs the taught theory with tested code and full documentation.

CHAPTER 3 · METHODOLOGY

Tools & Technology Stack

Python 3.10+

Core implementation language

PyTorch

Tensors, autograd, training

tiktoken

GPT-2 byte-pair tokenizer

Jupyter Notebook

Experiments & demos

pytest

Automated testing

Git & GitHub

Version control

VS Code

Development environment

CUDA GPU / CPU

Training compute

CHAPTER 3 · SYSTEM DESIGN

System Architecture

- Accepts raw input text and converts it into tokens.
- Generates token and positional embeddings for contextual representation.
- Processes embeddings through **12 Transformer layers** with causal self-attention.
- Uses feed-forward networks, layer normalization, and residual connections to learn features.
- Produces output logits to predict the next most probable token.
- Accepts raw input text and converts it into tokens using **Tiktoken BPE**.
- **Dropout** is applied during training to improve generalization and reduce overfitting.
- The final representation is projected to the vocabulary space to generate **output logits**.

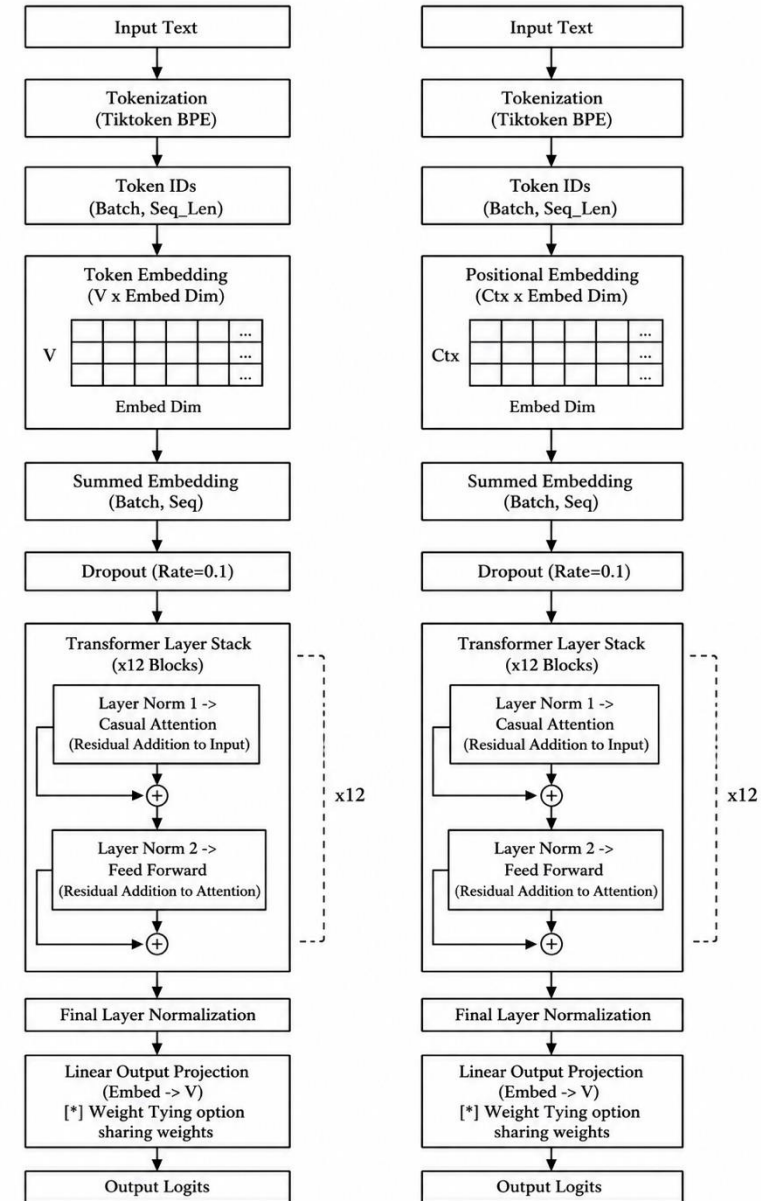


Figure 1: Proposed System Architecture

CHAPTER 3 · SYSTEM DESIGN

Use Case Diagram

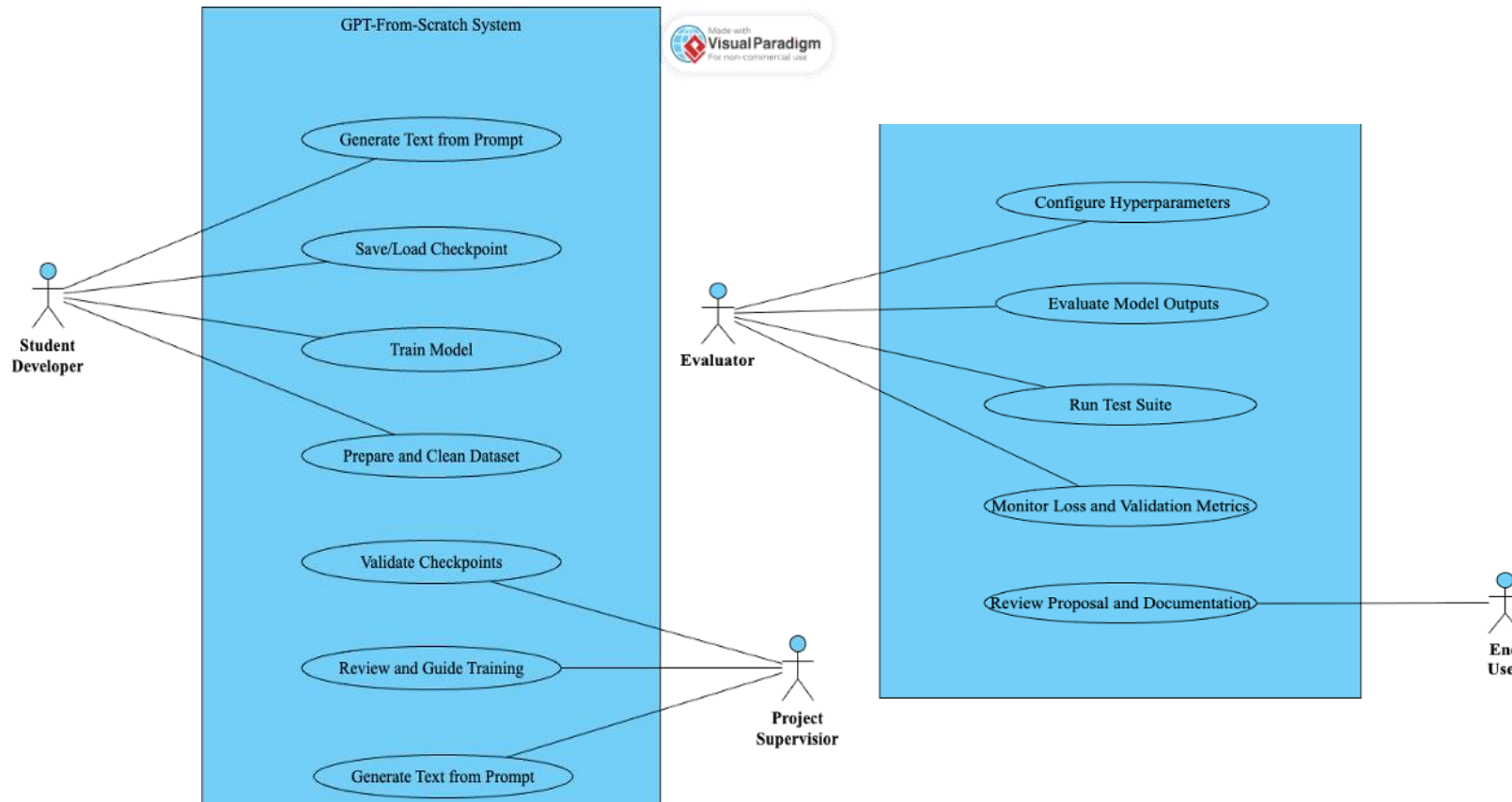


Figure 2: Use Case Diagram

Four actors interact with the system: the student developer, project supervisor, evaluator/panel, and end user.

Nine use cases span dataset prep, training, checkpointing, testing, generation, and documentation review.

CHAPTER 3 · SYSTEM DESIGN

ER Diagram – Experiment Tracking

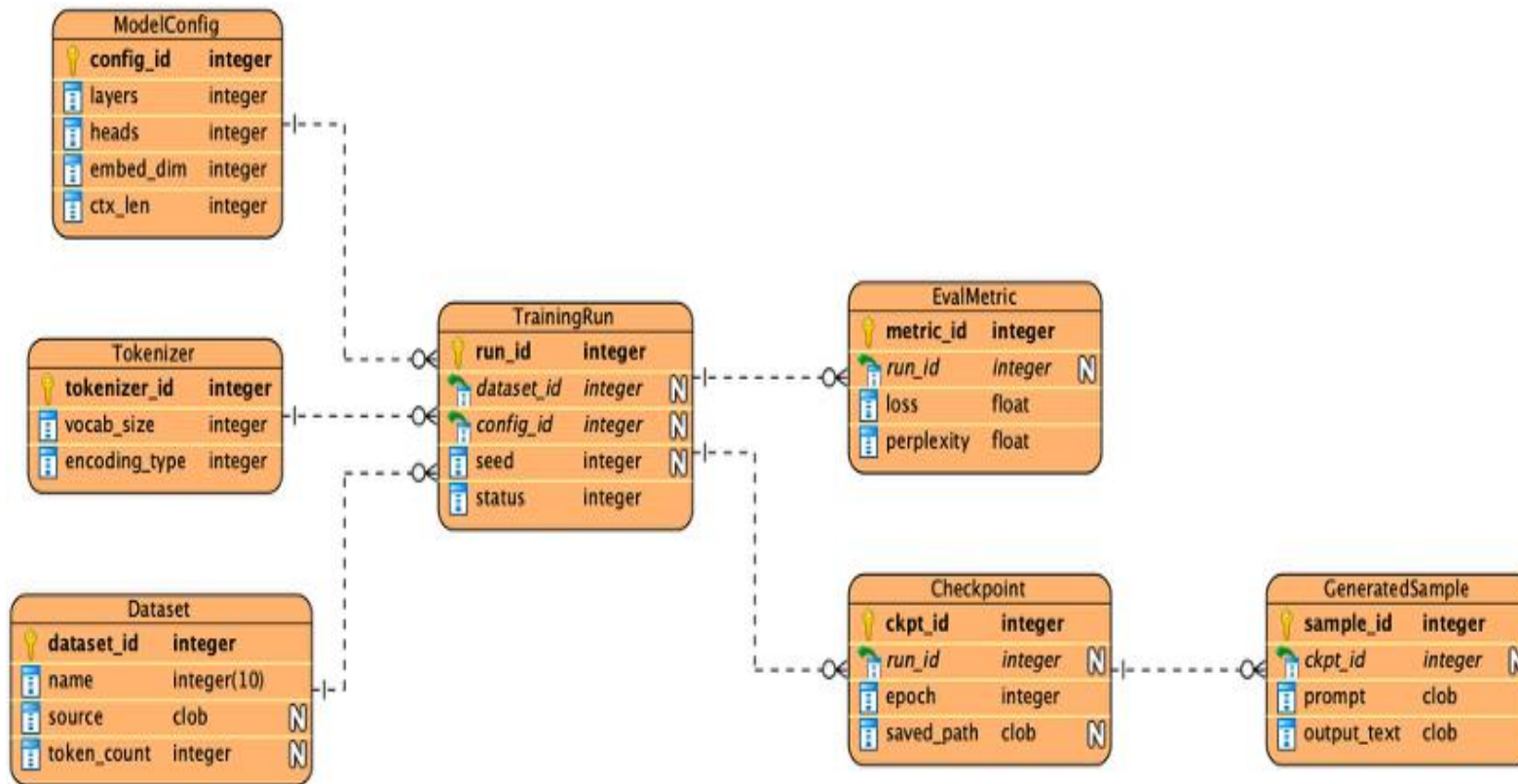


Figure 3: ER Diagram for Experiment Tracking

A lightweight schema (Dataset, Tokenizer, ModelConfig, TrainingRun, Checkpoint, EvaluationMetric, GeneratedSample) makes every result traceable to the exact configuration and data that produced it.

CHAPTER 3 · SYSTEM DESIGN

Sequence Diagram – Text Generation

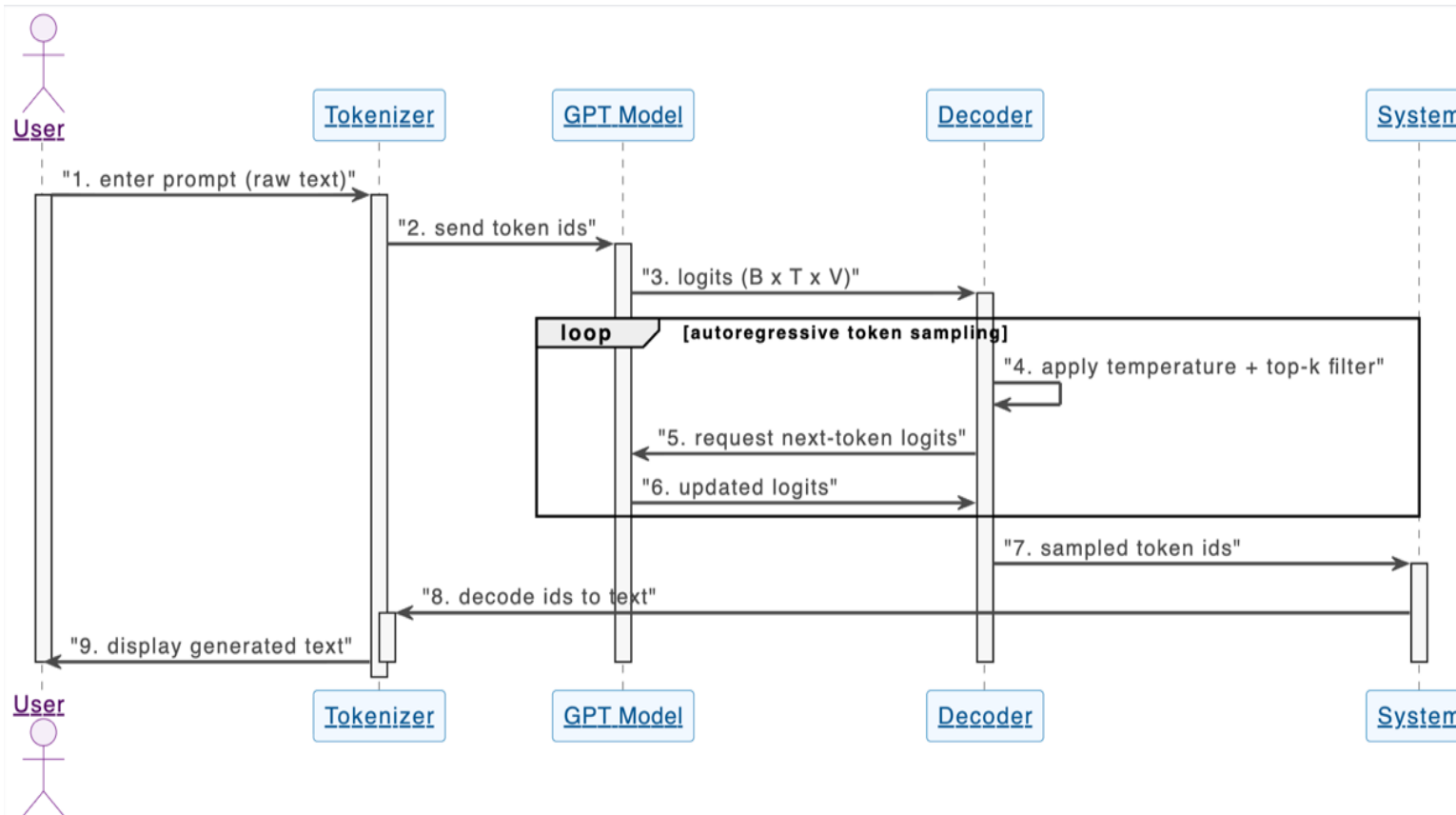


Figure 4: System Sequence Diagram – Text Generation

A prompt is tokenized, passed through the model, filtered by temperature and top-k, sampled autoregressively in a loop, then decoded back into readable text.

CHAPTER 3 · TOOLS AND METHODOLOGY

Sampling Strategies

Three decoding strategies were implemented and compared for text generation.

Greedy

Always picks the highest-probability token — fully deterministic, no randomness.

Temperature Scaling

Divides logits by temperature before softmax; lower T sharpens, higher T adds randomness.

Top-k Sampling

Keeps only top-k tokens, renormalizes, and samples via a multinomial distribution.

CHAPTER 3 · SYSTEM DESIGN

Sequence Diagram – Training Iteration

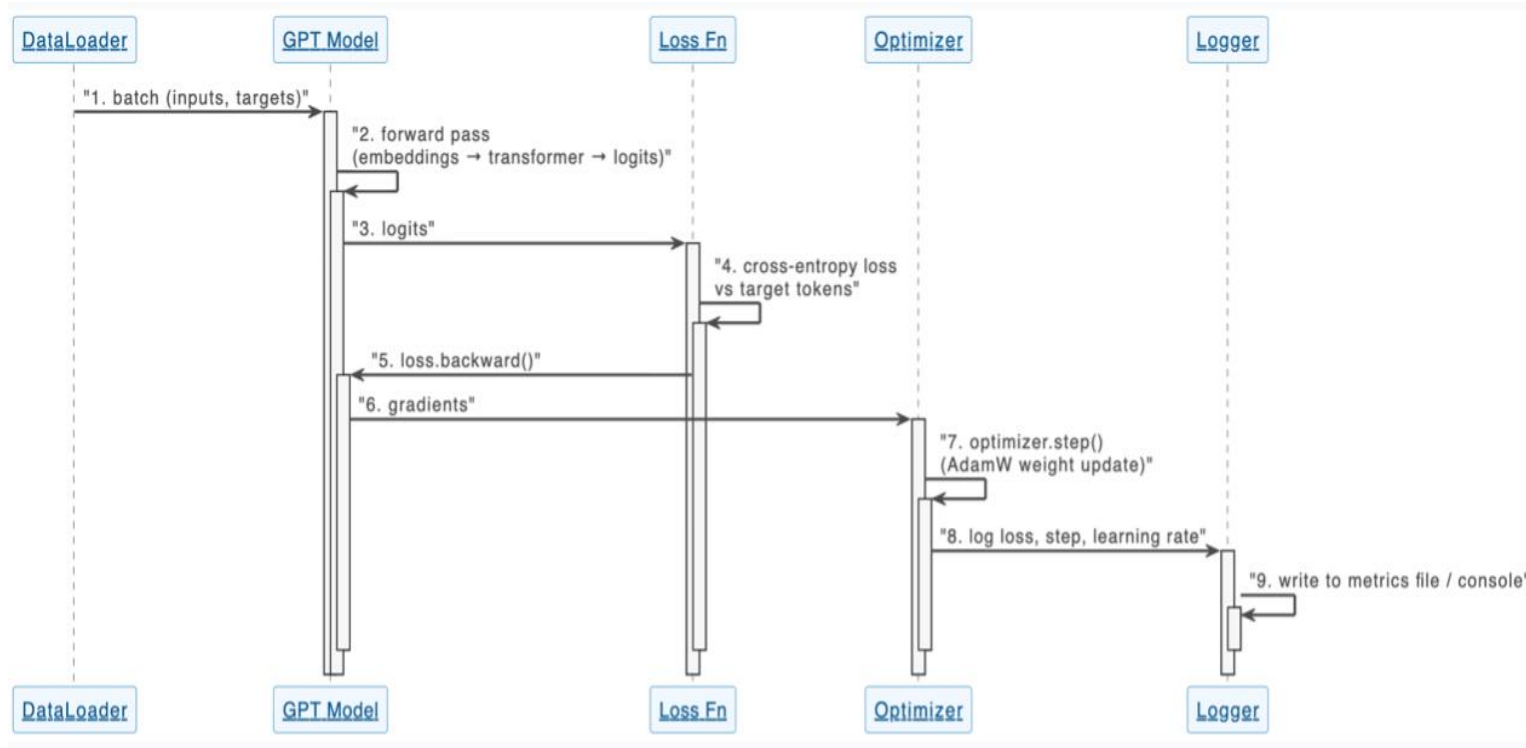


Figure 5: Sequence Diagram – One Training Iteration

One training step: batch
→ forward pass →
cross-entropy loss →
backward pass →
AdamW update → metric
logging.

CHAPTER 3 · SYSTEM DESIGN

Design Class Diagram

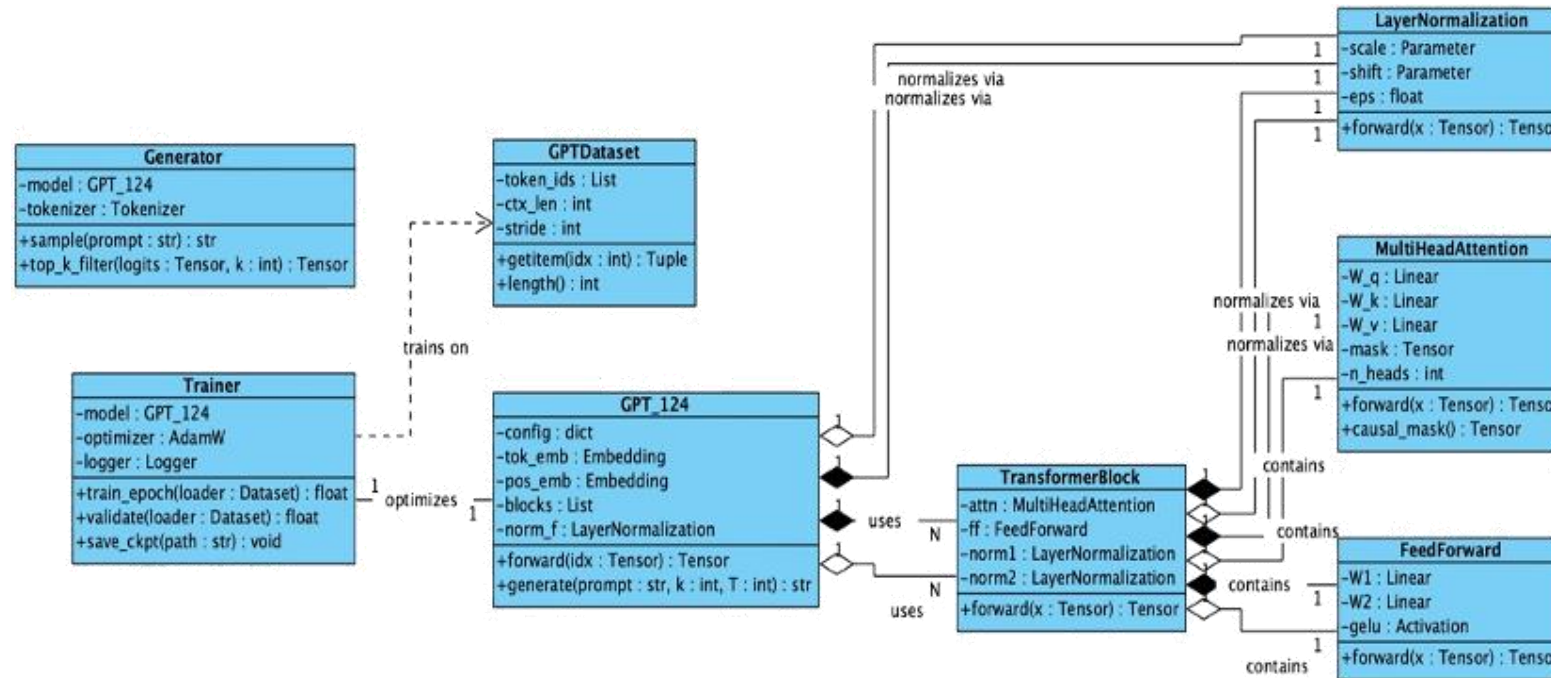
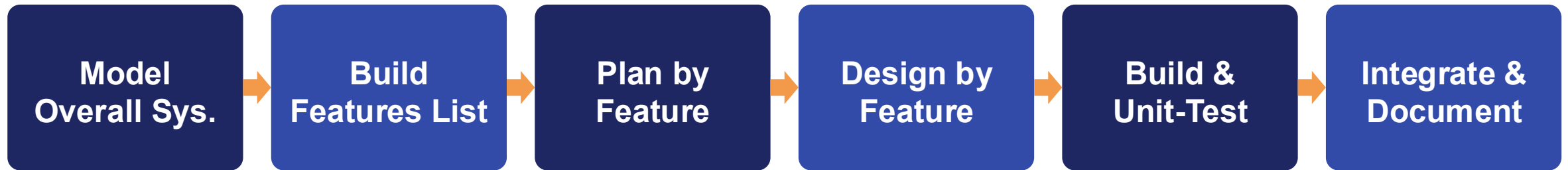


Figure 6: Design Class Diagram

GPT_124 composes a stack of TransformerBlocks, each containing MultiHeadAttention and FeedForward, normalized via LayerNormalization. Trainer and Generator wrap the training and inference workflows.

CHAPTER 3 · TOOLS AND METHODOLOGY

Feature Driven, Iterative Development



Each feature – tokenizer dataset, transformer block, training loop, evaluation, generation – is designed, implemented, unit-tested, and only then integrated. This keeps debugging local and produces a demonstrable increment every week.

- Tokenizer pipeline
- Dataset pipeline
- Transformer block
- GPT model
- Training loop
- Evaluation
- Generation
- Testing & docs

CHAPTER 3 · TOOLS AND METHODOLOGY

Project Structure

A modular layout separates architecture, data, training, and inference into independent pieces.

Core Modules

model/ for architecture, data/ for the dataset and loader, configs/ for hyperparameters.

Pipeline Modules

training/ for the training loop and losses, inference/ for text generation and sampling.

Support

utils/ for metrics and plotting, plus a one-command demo script for reproducible runs.

CHAPTER 3 · TOOLS AND METHODOLOGY

Team Roles & Data Methods

Responsibility rotates by feature, not by person — every member can read, run, and explain any layer of the system, from tokenizer to decoding.

Team Roles

Shared-ownership model: whoever picks up a feature owns its code, tests, and docs.

Data Collection

Small, clearly licensed or public-domain corpora, starting with the repository's sample text.

Data Analysis

Loss & perplexity tracked every eval interval; samples reviewed alongside the metrics.

Ethical & Responsible-Use Considerations

Even a small, educational language model raises questions worth addressing explicitly, rather than leaving implicit.

Corpus Licensing

Only small, clearly licensed or public-domain text is used — no scraped or unlicensed corpora.

Honest Reporting

Outputs are shown as a demonstration of mechanism, not as factually reliable text.

Known Limitations

Repetition and unreliability are documented alongside results, not hidden behind cherry-picks.

CHAPTER 4 · RESEARCH PLAN AND BUDGET

Testing Plan

Test Case	What It Verifies
Tokenizer Test	Encode/decode round-trip is correct
Dataset Shape Test	Tensor shapes match batch × context length
Attention Mask Test	No position attends to the future
Forward Pass Test	Output shape = batch × seq × vocab
Loss Test	Loss is finite, decreases on overfit sample
Generation Test	Output respects length & sampling limits
Checkpoint Test	Reloaded model reproduces identical logits

CHAPTER 4 · RESEARCH PLAN AND BUDGET

8-Week Work Schedule

Tasks:

1. Requirement study and repository review
2. Literature review and dataset/tokenizer study
3. Dataset pipeline and data loader refactor
4. Model Implementation review and shape tests
5. Training loop, checkpointing, logging
6. Evaluation, decoding experiments, risk review
7. Unit testing and integration testing and Final report, presentation and demo preparation

Tasks	Weeks							
1	1							
2		2						
3			3					
4				4				
5					5 and 6			
6							7	
7								8

Figure 7: Project Gantt Chart

CHAPTER 4 · RESEARCH PLAN AND BUDGET

Risk Matrix & Budget

Impact	High	Compute / GPU limitation	Training instability	
	Medium	Overfitting on small corpus		Time Overrun near deadline
	Low	Checkpoint / data loss	Tokenizer edge class	
		Low	Medium	High
		Likelihood		

Figure 8: Risk Matrix

GPU/compute limits

Gradient accumulation, cloud fallback

Training instability

Grad clipping, LR warmup, frequent checkpoints

Overfitting

Validation split, dropout, monitoring

Time overrun

Weekly schedule, continuous documentation

Budget: No dedicated budget required — open-source tools, personally-owned hardware, free-tier cloud compute.

CHAPTER 5 · EXPECTED OUTCOME

Model Training Results

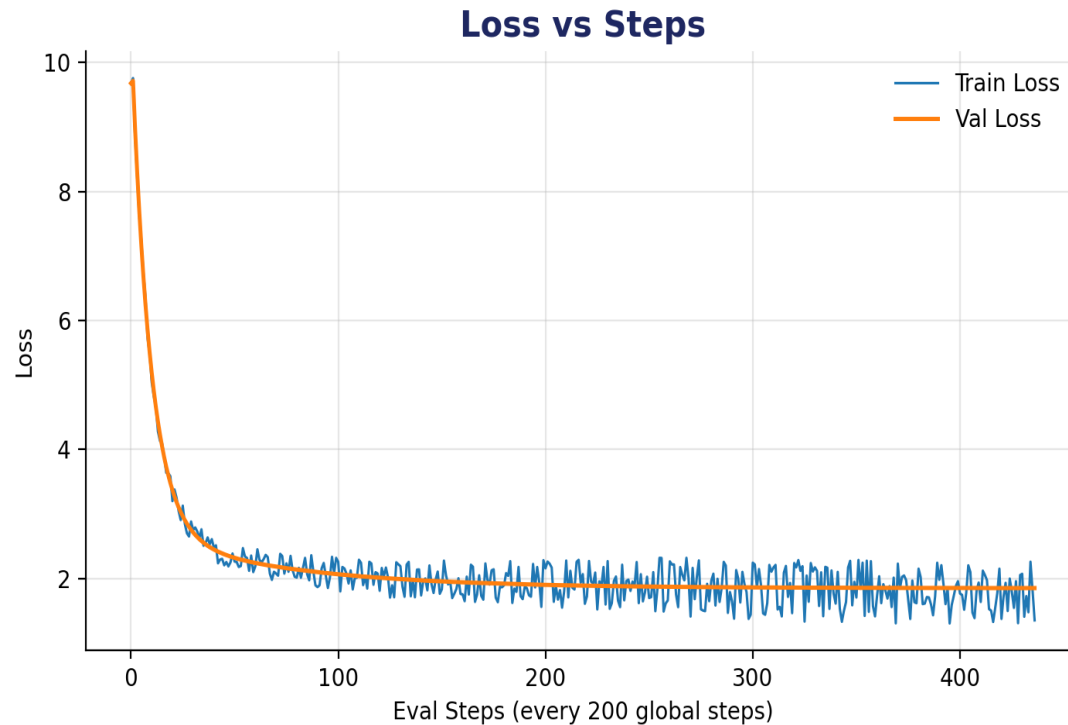


Figure 9: Loss vs Steps

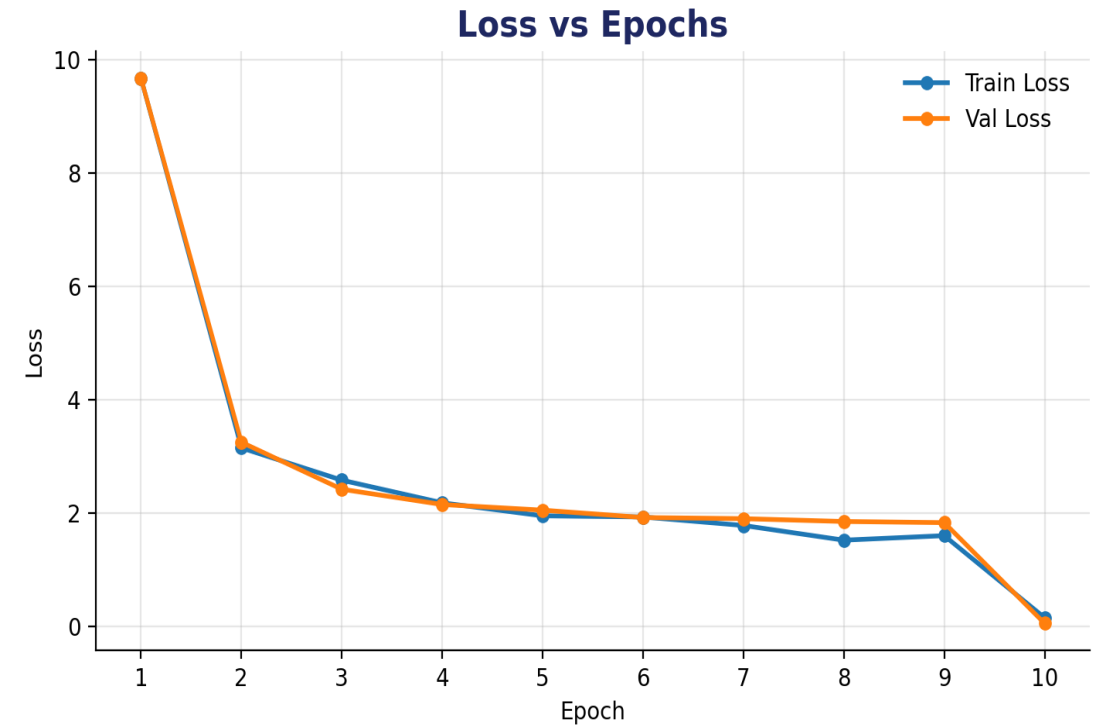


Figure 10: Loss vs Epochs

CHAPTER 5 · EXPECTED OUTCOME

Generated Text Examples (Illustrative)

Placeholders shown here — replace with real generations once training completes.

Prompt → Output

“Once upon a time” → a short, coherent story continuation in the corpus style.

Sampling Used

temperature = 0.8, top_k = 40 — balances coherence with controlled variation.

Status

Illustrative only — actual samples will be substituted once the model is trained.

CHAPTER 5 · EXPECTED OUTCOME

Expected Deliverables

Working, end-to-end GPT-style model trainable on a chosen corpus

Documented, tested tokenizer & dataset pipeline

Training/validation logs showing visible learning

Generated samples under temperature & top-k sampling

Reproducible repo: config logging, checkpoints, tests, CLI

Complete proposal, mid-defence report, final report & slides

CHAPTER 5 · EXPECTED OUTCOME

Contributions

Modular code structure separating tokenizer, dataset, model, training, and generation

Automated test suite: shapes, causal masking, loss, checkpointing, generation

Documented evaluation protocol with tracked loss & perplexity curves

Complete design artifacts: architecture, use case, ER, sequence & class diagrams

Explicit feasibility, risk, and ethical-use analysis the original repository lacks

A disciplined engineering treatment of an existing model, not a new architecture

CHAPTER 5 · EXPECTED OUTCOME

Future Works

Cross-lingual extension: repoint the pipeline at a Nepali-language corpus

Empirical scaling study: vary depth & width, observe loss-vs-parameters

Lightweight web interface for interactive prompting & decoding comparison

Instruction tuning on a small instruction-following dataset

Reuse as a teaching artifact for future Research Fundamentals cohorts

Continued documentation of model evaluation, bias & hallucination behaviour

CHAPTER 5 · EXPECTED OUTCOME

Deployment (Planned Extension)

An optional future-work extension — not part of the core proposal's deliverables or claims.

Interface

A lightweight Streamlit app with temperature and top-k sliders for interactive prompting.

Hosting

Free-tier hosting (e.g. HuggingFace Spaces) — no dedicated budget or infrastructure required.

Status

Listed under Future Works (Section 5.6) — not a deliverable of this proposal.

Thank You

Repository: github.com/beeks-code/GPT-From-scratch

Bikram Sharma • Pramesh Lamichhane • Swikar Poudel